Machine Learning for Data Science

Learn • Build Models • Make Predictions • Solve Real-World Problems

From data to intelligent decisions!

1. Introduction to Machine Learning

- Types of Learning: Supervised, Unsupervised, Reinforcement Learning
- The Machine Learning workflow

Types of learning in ML

Types of Learning

Supervised (uses labeled data)
e.g. classification, regression

Unsupervised (uses unlabeled data)
e.g. clustering, dimensionality reduction

Reinforcement (learns by rewards)
e.g. game playing, robotics

Data → Prepare → Train → Evaluate → Predict

2. Data Preparation for ML

- Feature Engineering
- Feature Scaling
- Train-Test Split
- Cross-Validation

Prepare data for better models

Feature Engineering

new features from existing data

Age	Income	BMI
25	50000	22.5
...	...	...

Feature Scaling

normalize / standardize

$x' = \frac{x - \mu}{\sigma}$

before after

Train-Test Split

80% train 20% test

Train | Test

Cross-Validation

k-folds

Fold 1 Fold 2 Fold 3 ...

3. Supervised Learning

- Linear Regression
- Logistic Regression
- k-Nearest Neighbours
- Decision Trees

Predict from labeled data

Linear Regression

$y = mx + c$

Logistic Regression

$P(y=1)$ $\sigma(z) = \frac{1}{1+e^{-z}}$ 0 1 x

k-Nearest Neighbours

k = 3

Decision Trees

Root → Yes / No

4. Unsupervised Learning

- k-Means Clustering
- Hierarchical Clustering
- Dimensionality Reduction with PCA

Find patterns in unlabeled data

k-Means Clustering

Hierarchical Clustering

PCA (Dimensionality Reduction)

High-dimensional data → PCA → PC1, PC2 2D (reduced)

5. Model Evaluation

- Overfitting and Underfitting
- Confusion Matrix
- Accuracy, Precision, Recall, F1-Score
- ROC Curve

Measure model performance

Overfitting vs Underfitting

Underfitting Good Fit Overfitting

Confusion Matrix

	Predicted Positive	Predicted Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

Metrics

$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN}$

$\text{Precision} = \frac{TP}{TP+FP}$

$\text{Recall} = \frac{TP}{TP+FN}$

$\text{F1-Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$

ROC Curve

True Positive Rate 0 1 False Positive Rate

AUC = 0.85

6. Introduction to scikit-learn

- Building, training and evaluating ML models in Python

Build ML models with scikit-learn

```python
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

# Load data (example: breast cancer dataset)
from sklearn.datasets import load_breast_cancer
X, y = load_breast_cancer(return_X_y=True)

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Train model
model = LogisticRegression()
model.fit(X_train, y_train)

# Evaluate
pred = model.predict(X_test)
print(accuracy_score(y_test), pred)
print(classification_report(y_test, pred))
```

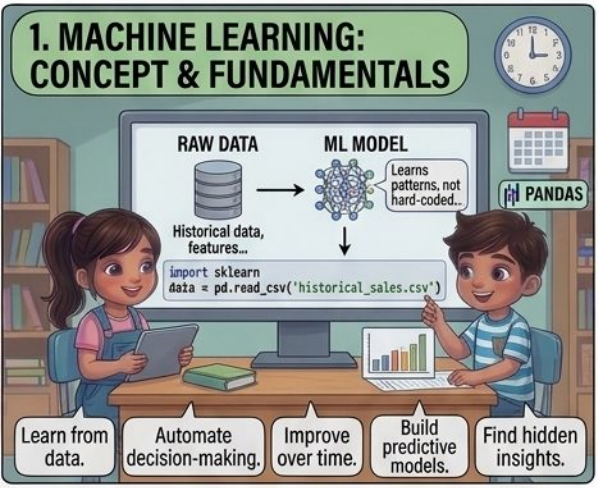
Train and evaluate model

Data → Prepare → Train → Evaluate → Get Insights

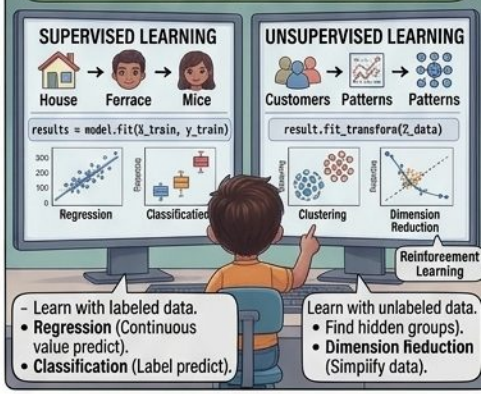
Better models = Better decisions!

PYTHON: MACHINE LEARNING FOR DATA SCIENCE: INTRODUCTION TO MACHINE LEARNING

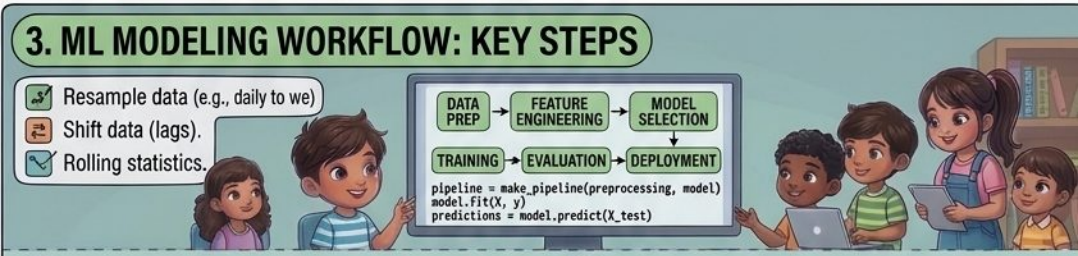
1. MACHINE LEARNING: CONCEPT & FUNDAMENTALS



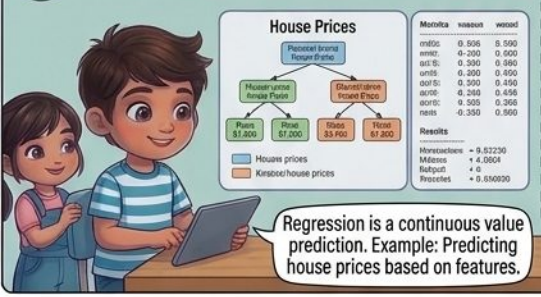
2. MACHINE LEARNING TYPES & LEARNING PARADIGMS



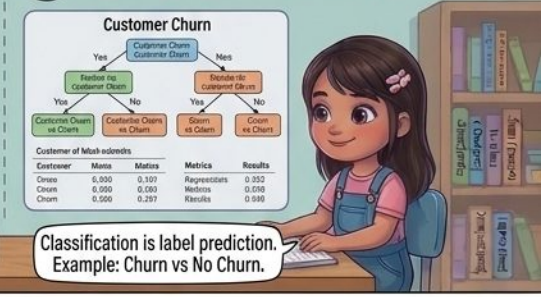
3. ML MODELING WORKFLOW: KEY STEPS



3a. SUPERVISED REGRESSION MODEL



3b. CLASSIFICATION MODEL



4. REAL-WORLD APPLICATIONS & CASE STUDIES

Recommender Systems

`recommendation_engine.recommend(user_id)`



Financial Risk Assessment

`risk_model.predict_probability(loan_application)`



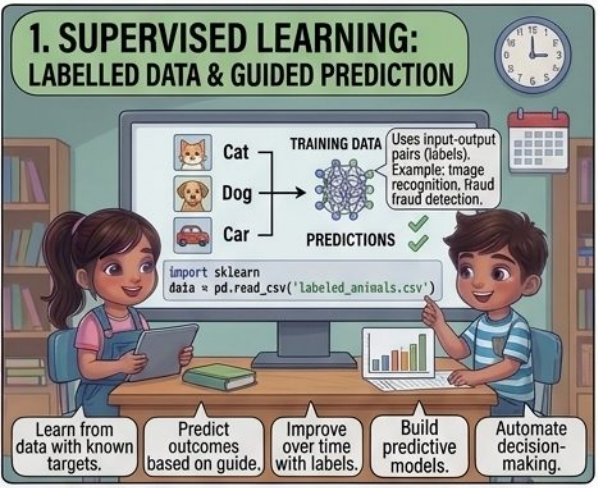
Healthcare Diagnosis

`diagnostic_model.classify(medical_image)`

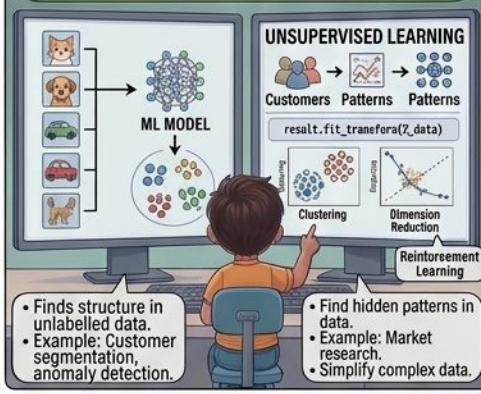


PYTHON: TYPES OF LEARNING: SUPERVISED, UNSUPERVISED AND REINFORCEMENT LEARNING

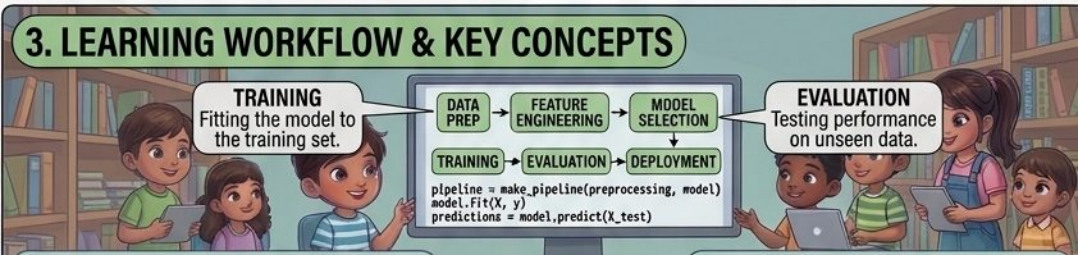
1. SUPERVISED LEARNING: LABELLED DATA & GUIDED PRIDICTION



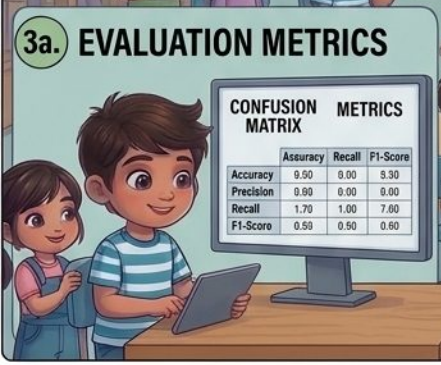
2. UNSUPERVISED LEARNING: UNLABELLED DATA & PATTERN DISCOVERY



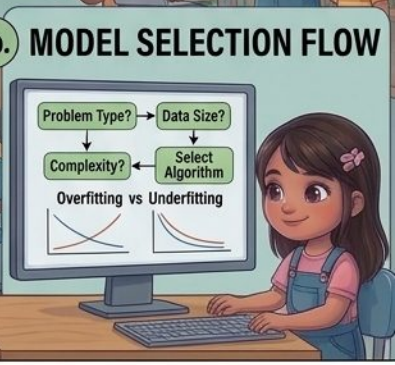
3. LEARNING WORKFLOW & KEY CONCEPTS



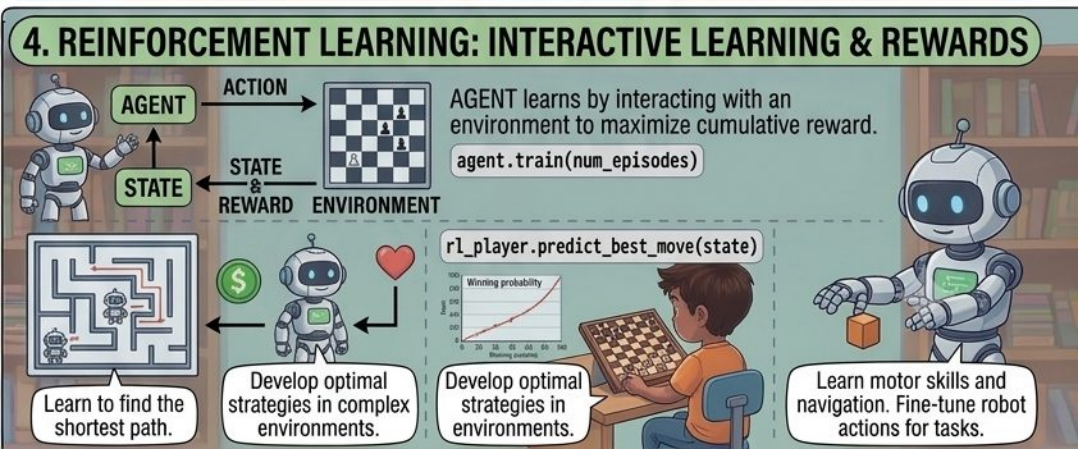
3a. EVALUATION METRICS



3b. MODEL SELECTION FLOW

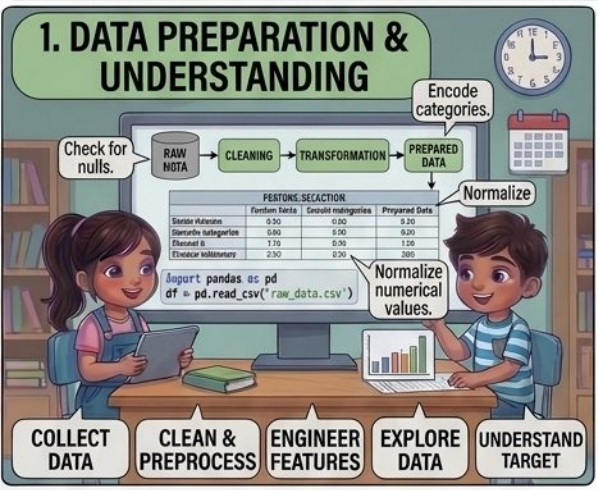


4. REINFORCEMENT LEARNING: INTERACTIVE LEARNING & REWARDS

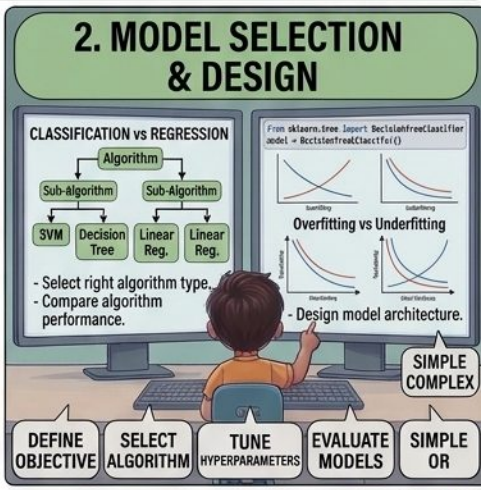


THE MACHINE LEARNING WORKFLOW: FROM DATA TO DEPLOYMENT

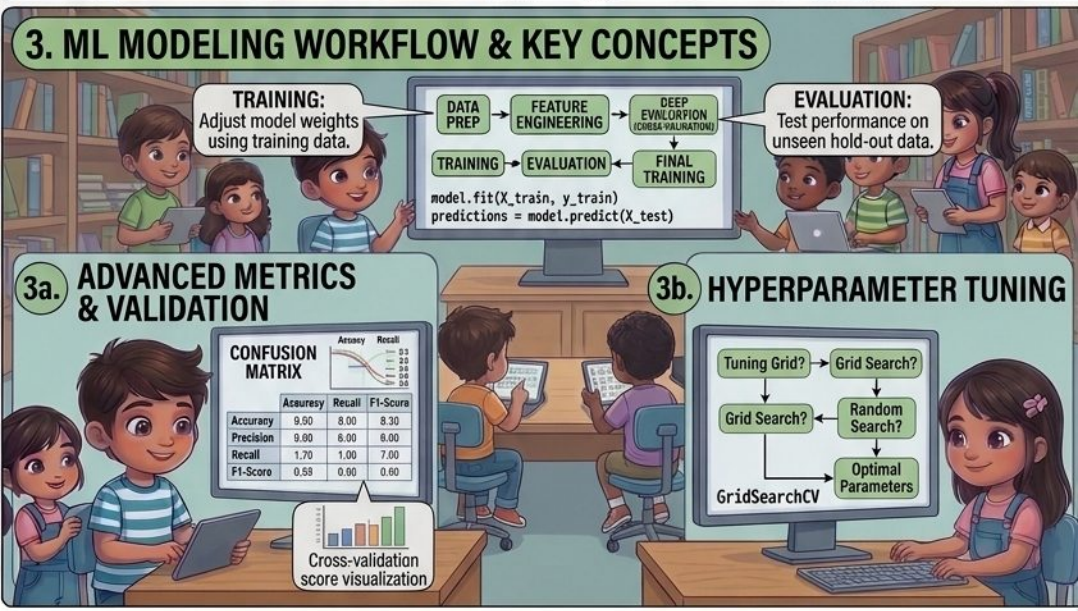
1. DATA PREPARATION & UNDERSTANDING



2. MODEL SELECTION & DESIGN



3. ML MODELING WORKFLOW & KEY CONCEPTS

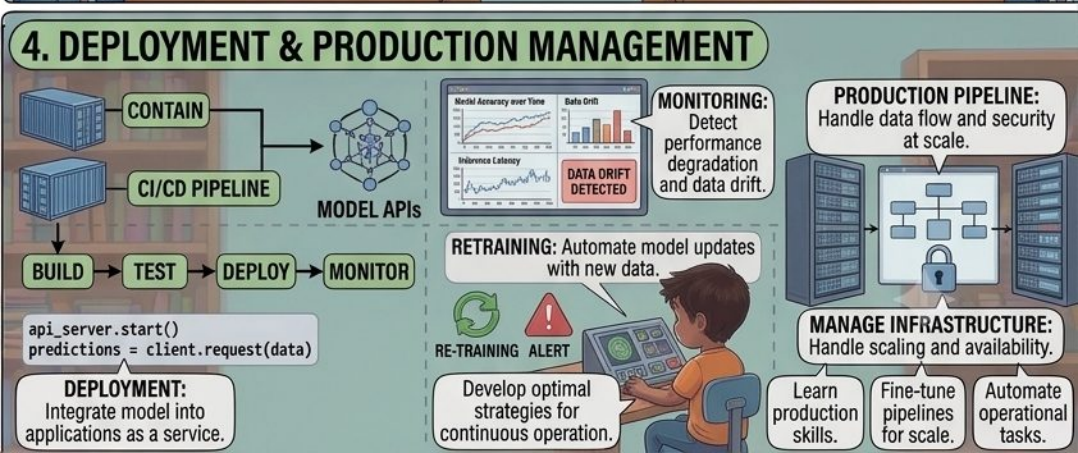


3a. ADVANCED METRICS & VALIDATION

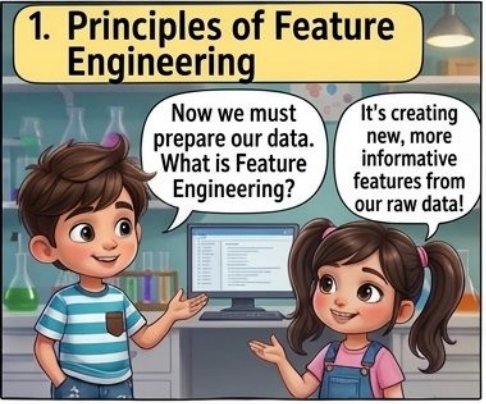
	Accuracy	Recall	F1-Score
Accuracy	9.90	8.00	8.30
Precision	9.90	6.00	6.00
Recall	1.70	1.00	7.00
F1-Score	0.58	0.00	0.60

3b. HYPERPARAMETER TUNING

4. DEPLOYMENT & PRODUCTION MANAGEMENT



1. Principles of Feature Engineering



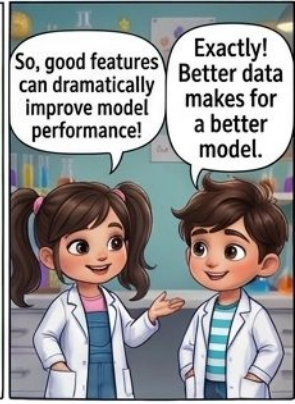
Raw data

timestamp	full_name	price
2021-05-09	Red	14
2021-03-07	full_name	5
2021-03-26	price	100
2021-03-22	price	3.5
...	...	...

Engineered features

- day_of_week
- first_name
- log_price

```
import pandas as pd
df['day_of_week'] = df['timestamp'].dt.dayofweek
df['log_price'] = np.log(df['price'])
```



2. Categorical Encoding Techniques



A One-Hot Encoding (for unordered data)

Input	Output	Output
Red	1 0	0 0
Blue	0 1	1 0
Green	0 1	0 1

B Ordinal Encoding (for ordered data)

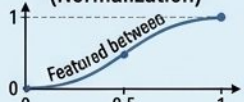
Input	Output
Low	0
Medium	1
High	2



3. The Importance of Feature Scaling



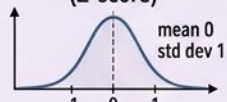
A Min-Max Scaling (Normalization)



$$x_{\text{scaled}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

```
from sklearn.preprocessing import MinMaxScaler
```

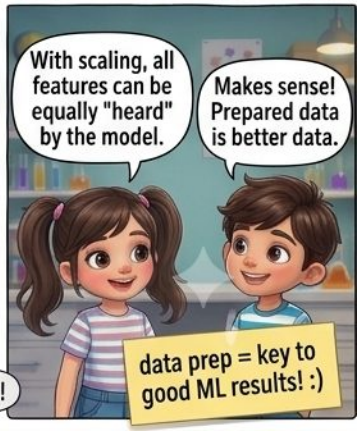
B Standardization (Z-score)



$$x_{\text{std}} = \frac{x - \mu}{\sigma}$$

```
from sklearn.preprocessing import StandardScaler
```

Prevents large features from dominating smaller ones!





TRAIN-TEST SPLIT AND CROSS-VALIDATION

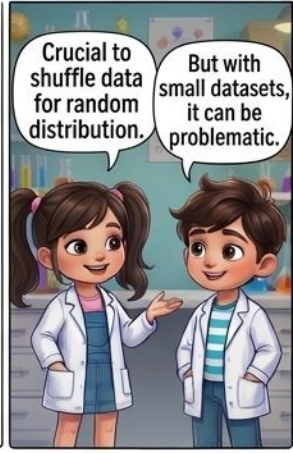
1. Basic Train-Test Split (The Data Shuffle)

Raw data		
Features	1-3	Target
	...	1
	...	2
...	...	3
...	...	4
...	...	5

80% → TRAINING SET — FOR MODEL TRAINING

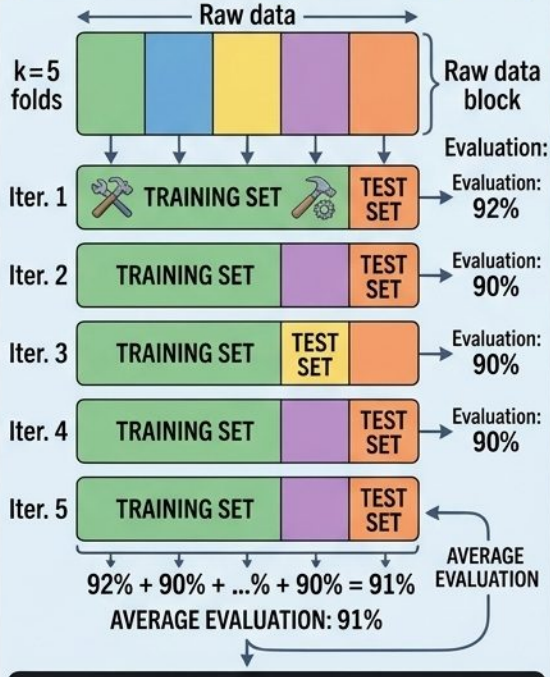
20% → TEST SET — FOR MODEL EVALUATION

```
import pandas as pd
X, y = df.data, df.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```





2. The k-Fold Cross-Validation Approach




```
from sklearn.model_selection import cross_val_score
scores = cross_val_score(model, X, y, cv=5)
# print(scores.mean())
```





Cross-Validation Variations

- **Stratified k-Fold:** Preserves label class proportions in each fold.



- **Repeated k-Fold:** Repeats k-Fold n times for an even more stable estimate.



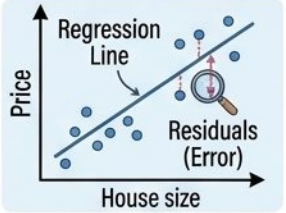
Correct evaluation = key to model performance! :)

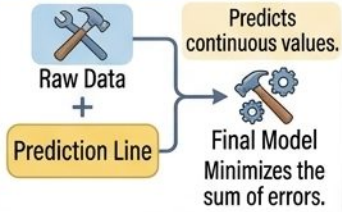
SUPERVISED LEARNING – LINEAR REGRESSION

We need a way to predict numerical values. Like house prices! How?

We use Linear Regression! It finds a trend line.

1. The Concept of Linear Regression: Line of Best Fit






```
import pandas as pd
from sklearn.linear_model import LinearRegression
model.fit(X_train, y_train)        y_pred = model.predict(X_test)
```

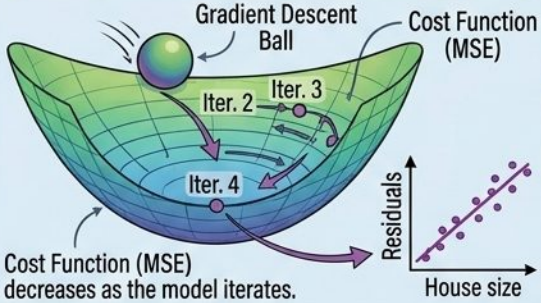
So a good line can make accurate predictions!

Exactly! Minimizing errors is the key to a better model.

But how does the computer actually find that exact line?

By optimizing the model! It's like walking down a mountain.

2. Finding the Best Line: Gradient Descent Approach



1. Calculate Gradient (Direction) → 2. Update Weights → 3. Repeat (Walk Down Hill)

Every iteration makes the model slightly better.

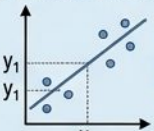
Provides a stable way to find the optimal weights!

Are there simple equations we should know?

Yes, for simple cases!

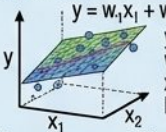
3. Key Equations & Concepts

(A) Simple Linear Regression



y=predicted value, feature variables. to b e=intercept.

(B) Multiple Linear Regression



Uses one or multiple feature variables. The model must balance complexity.

```
print(f"Intercept: {}")
print(f"Coefficient(s):
{model.coef_}")
```

Correct prediction = key to good results! :)

Choose the right variables and minimize cost!

Correct optimization is key to a good model!

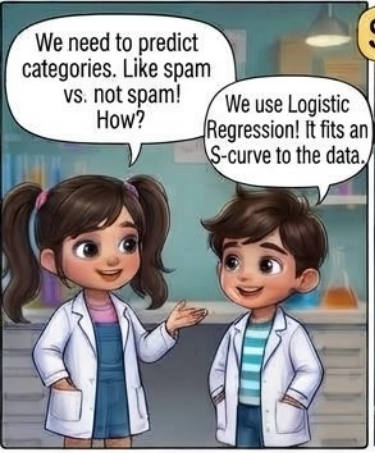


- Choose your features carefully: and check for multicollinearity!

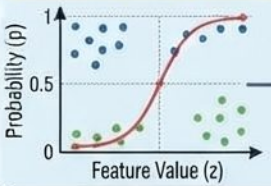


Good model fit = key to good results! :)

SUPERVISED LEARNING – LOGISTIC REGRESSION



1. The Concept of Logistic Regression: Sigmoid Curve and Probability



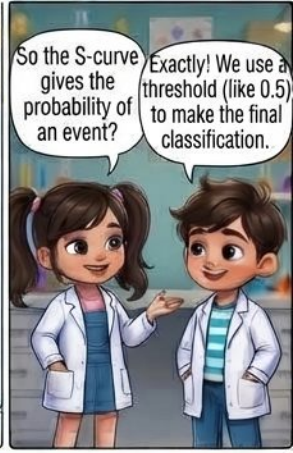
$z = w \cdot x + b$

Sigmoid Function $\sigma(z)$

Predicts probability of a class (0 or 1).

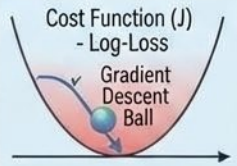
Outputs mapped to (0, 1) range.

```
import numpy as np
from sklearn.linear_model import LogisticRegression
model.fit(X_train, y_train)
y_pred_prob = model.predict_proba(X_test)[:,1] # Probability of Class 1
```



2. Training the Model: Log-Loss (Cross-Entropy) and Optimization





Goal: Minimize Log-Loss (Cost).

Log-Loss penalizes confident wrong answers.

1. Calculate Gradient → 2. Update Weights → 3. Repeat (Follow the Slope)

Log-Loss penalizes confident wrong answers.

$\hat{y}$ is the predicted probability

$$L(\hat{y}, y) = -[y \log(\hat{y}) + (1-y) \log(1-\hat{y})]$$

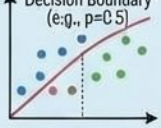
where $\hat{y}$ is the predicted probability



3. Key Concepts and Evaluation



(A) Simple Logistic Regression
(2 features, straight decision boundary on the feature plane)



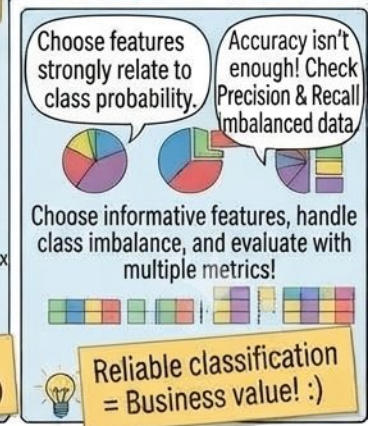
(B) Multiple Logistic Regression
(many features, non-linear decision boundary)

Actual Class	Predicted Class 0	Predicted Class 1
0	True Negative	False Positive
1	False Negative	True Positive

```
from sklearn.metrics import accuracy_score, confusion_matrix
y_pred = model.predict(X_test)
print(accuracy_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
```

```
print(f"Intercept: {}")
print(f"Coefficient(s): {model.coef_}")
```

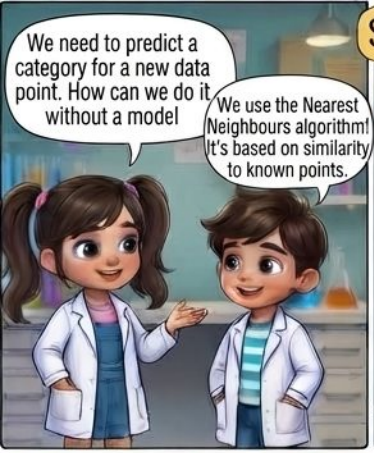
Correct prediction = key to good results! :)



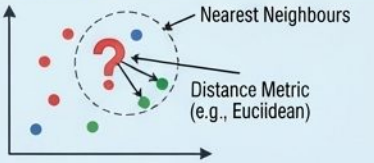
Choose informative features, handle class imbalance, and evaluate with multiple metrics!

Reliable classification = Business value! :)

SUPERVISED LEARNING – NEAREST NEIGHBOURS (k-NN)



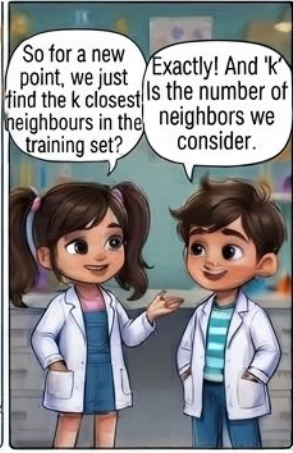
1. The Concept of k-NN: Instance-Based Learning



Uses the 'closest' labeled examples to classify a new query.

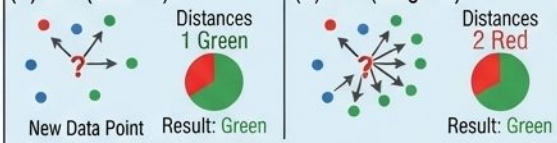
No explicit model is trained.

```
import numpy as np
from sklearn.neighbors import KNeighborsClassifier
# X_train: features, y_train: labels
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)
```





2. Making a Prediction: Distance and Voting



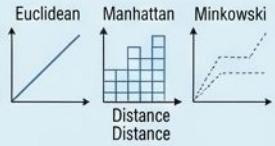
The class label is determined by a majority vote (Classification) or average (Regression). Distance weighting can be applied.

$$d(p, q) = \sqrt{\sum (p_i - q_i)^2}$$



3. Key Considerations & Evaluation

Distance Metrics



Curse of Dimensionality: Performance degrades with many irrelevant features.

Distance Metric Choice

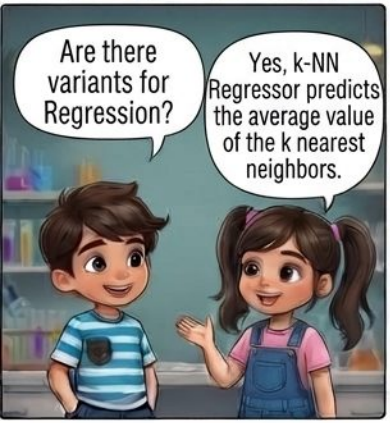
Euclidean

Manhattan

Confusied Matrix

Actual Class \ Predicted Class	A	B
A	A	B
B	A	B

```
from sklearn.model_selection import cross_val_score
# Cross-Validation for k-NN
scores = cross_val_score(knn, X, y, cv=5)
print(f"Accuracy: an k-NN")
print(f"Accuracy: {scores.mean():.2f}")
print(f"Accuracy: {scores.mean():.2f}")
```



- [x] Choose appropriate k.
- [ ] Select a distance metric.
- [x] Scale features (crucial!).
- [ ] Consider feature selection.
- [ ] Check for computational cost (prediction is slow).

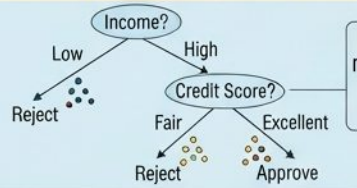




SUPERVISED LEARNING – NBJ – DECISION TREES

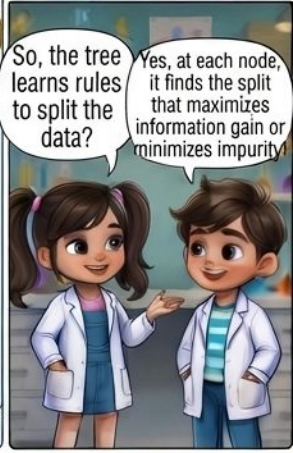


1. The Concept of Decision Trees: Tree Structure & Splits



Goal: Create pure leaf nodes (all same class). Uses rules based on feature values.

```
import numpy as np
# X_train: features, y_train: labels
dt = DecisionTreeClassifier(criterion='gini', max_depth=3)
dt.fit(X_train, y_train)
prediction = dt.predict(new_data)
```





2. Splitting Criteria: Impurity Reduction

(A) Gini Impurity

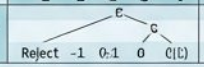


$$Gini(t) = 1 - \sum_{i} P(i|t)^2$$

(B) Entropy / Information Gain

$$Entropy(t) = -\sum_{i} P(i|t) \log_2 P(i|t)$$

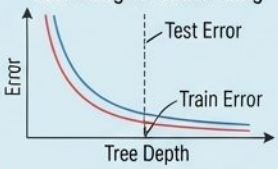
These measures tell us how 'mixed' the data is in a node. The best split minimizes impurity in children.

	1 1 2 3 0	Gini Reduction
Example Toxe		



3. Key Considerations & Evaluation

Overfitting vs Underfitting



Choose impurity measure (Gini or Entropy). Set pruning (max_depth, min_samples_split). Visualize the tree., 'Debt'))

Confused Matrix

Actual Class \ Predicted Class	A	B
A	A	B
B	A	B

```
from sklearn.tree import export_text
# Visualize the rules
print(f"Accuracy: r feature_names=['Income', 'Cre
print(f"Accuracy: {scores.mean(|..2f}")
print(r)
```



- [x] Choose impurity measure
- [ ] Set pruning parameters (max_depth, min_samples_s
- [ ] Visualize the tree for
- [ ] Consider ensemble methods (like Random Forests).



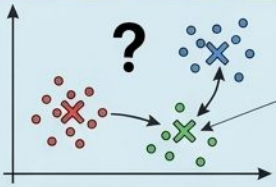


Decision Trees are powerful and interpretable but :)

UNSUPERVISED LEARNING – k-MEANS CLUSTERING

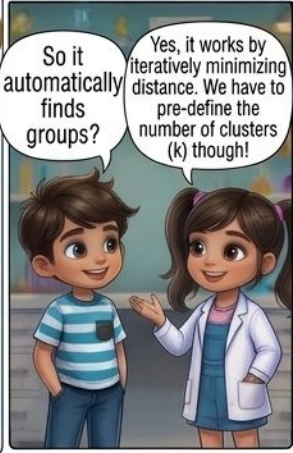


1. The Concept of k-Means Clustering: Finding Inner Structures



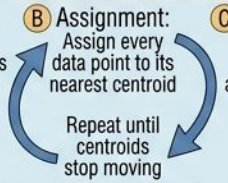
Goal: Find the inherent structures in unlabeled data. Divide data into k distinct groups (clusters) based on feature similarity.

```python
import numpy as np
from sklearn.cluster import KMeans
# X: Unlabeled data (features)
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans.fit(X)
predictions = kmeans.predict(X)
centroids = kmeans.cluster_centers_
```





2. The k-Means Algorithm: It's iterative!



Distance (Euclidean):

$$d(\mathbf{x}, c) = \sum_{c} (x_i - c_i)^2$$

Objective function:

$$WSS = \sum_{k} \sum_{cj} d(x_{ij}, c_j)^2$$

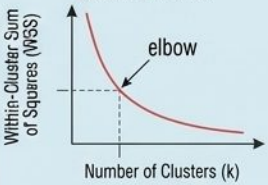
Minimize the intra-cluster distance and maximize the inter-cluster distance.





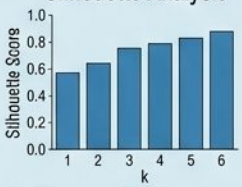
3. Key Considerations & Evaluation

Elbow Method



Use the Elbow Method to find the optimal k.

Silhouette Analysis



Evaluate with silhouette scores and cluster visualizations.

```python
from sklearn.metrics import silhouette_score
# Silhouette analysis to evaluate cluster quality
score = silhouette_score(X, kmeans.labels_)
print(f'Silhouette Score: {score:.2f}')
```

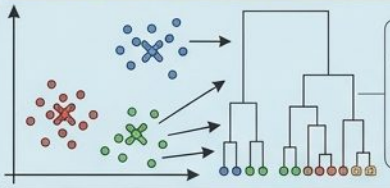
- [x] Pre-define k
- [ ] Choose distance metric (default: Euclidean)
- [x] Normalize data (important!)
- [ ] Run with multiple initializations (e.g., k-means++)
- [ ] Handle outliers

k-Means is efficient and easy to use, easy to use, but needs careful k selection and data prep! :)

UNSUPERVISED LEARNING – HIERARCHICAL CLUSTERING



1. The Concept of Hierarchical Clustering: Building a Tree



Goal: Create a hierarchy of nested clusters. Build a tree structure (dendrogram) showing similarities between all data points. No need to pre-define k initially.

```
import numpy as np
from scipy.cluster.hierarchy import dendrogram, linkage, fcluster
import matplotlib.pyplot as plt
# X: Unlabeled data (features)
Z = linkage(X, method='ward')
# Create fcluster assignments for specific cutoff
max_d = 7
clusters = fcluster(Z, max_d, criterion='distance')
```



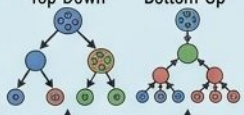


2. The Algorithm Types and Linkage Methods

A Algorithm Types:

Divisive vs Agglomerative

Top-Down Bottom-Up



Start with individual + merge, like this algorithm

B Linkage Methods

(Similarity measures between clusters)

Single Linkage: Distance between closest pair

$d(C_i, C_j) = \min d(x,y)$

Complete Linkage: Distance between farthest pair

$d(C_i, C_j) = \max d(x,y)$

Average Linkage: Average distance between all pairs

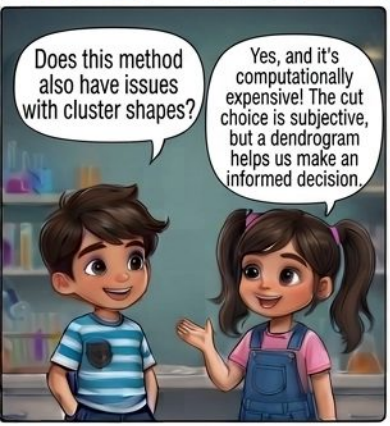
$d(C_i, C_j) = \min d(x,y)$

Ward's Method: Minimizes intra-cluster variance (like k-Means)

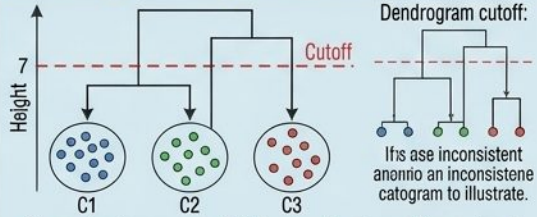
$d(C_i, C_j) = \min d(x,y)$

Choose the method to decide which two clusters to merge at each step. Agglomerative (Bottom-Up) is the most common





3. Dendrogram Analysis and Cutting the Tree



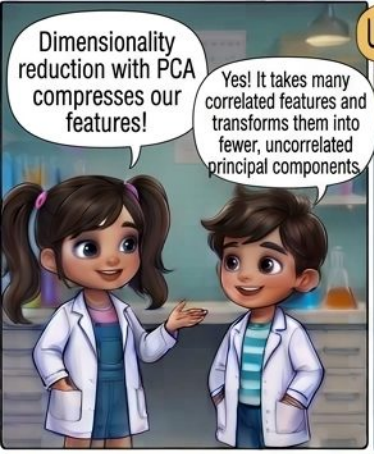
Use a dendrogram and `fcluster` to choose the optimal cut. Look for long vertical lines (representing a stable distance) and interpret the results.

```
from sklearn.cluster import AgglomerativeClustering
# Sklearn implementation for specific number of clusters
model = AgglomerativeClustering(n_clusters=3,
                affinity='euclidean', linkage='ward')
model.fit(X)
print(f'Cluster labels: {model.labels_}')
```

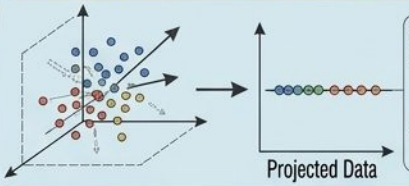
- [ ] Agglomerative or Divisive approach
- [x] Choose distance metric (default: Euclidean)
- [x] Choose linkage method (e.g., Ward, Complete)
- [ ] Handle large datasets ($O(n^2 \log n)$)
- [ ] Analyze dendrogram for stable cuts

Hierarchical Clustering is powerful for visualization but slow on large data and requires cut selection! :)

UNSUPERVISED LEARNING – DIMENSIONALITY REDUCTION WITH PCA

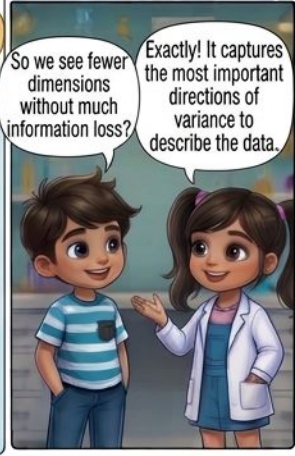


1. The Concept of PCA: Preserving Information with Principal Components



Goal: Map high-dimensional data into fewer dimensions. Preserve maximum data variance while removing Principal Components (PCs) become the new axes.

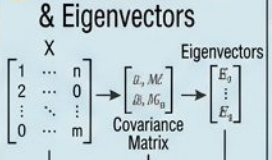
```python
import numpy as np
from sklearn.decomposition import PCA
# X: Correlated, high-dimensional data (features)
pca = PCA(n_components=2)
pca.fit(X)
reduced_data = pca.transform(X)
explained_variance = pca.explained_variance_ratio_
```



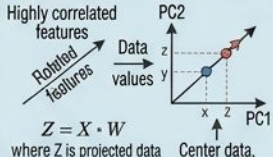


2. The Mathematical Approach and Key Steps

A Covariance Matrix & Eigenvectors



B Transformation Method



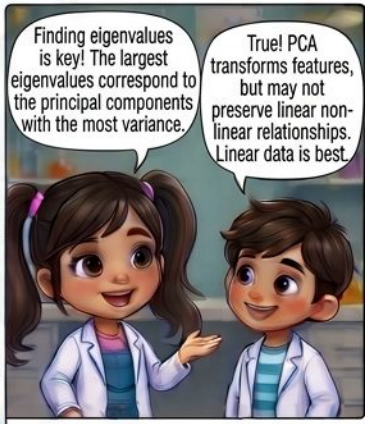
$$Z = X * W$$

where Z is projected data
W = matrix of eigenvectors

Center data, compute covariance, find ax and select top k, project data.

$$Z = X * W \longrightarrow$$

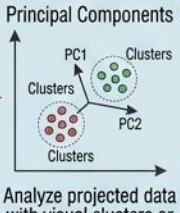
PCA is a linear transformation. We use eigenvectors of the covariance as the new basis, with the top k vectors chosen based on their eigenvalues.



3. Component Selection and Interpreting Results

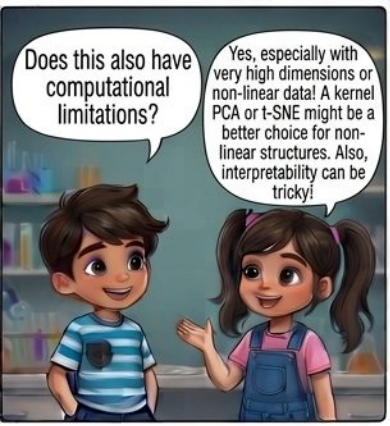


Use the Scree Plot to choose the number of components. Look for the 'elbow' to keep maximum variance with minimum features.



Analyze projected data with visual clusters or for subsequent tasks like classification.

```python
import matplotlib.pyplot as plt
# Scree plot analysis to choose components
pca = PCA()
pca.fit(X)
eigenvslues = pca.explained_variance_
plt.plot(np.arange(1, len(eigenvalues)+1), eigenvalues, marker='o')
plt.title('Scree Plot')
```



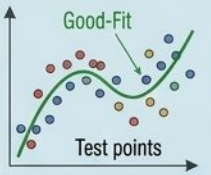
- ☐ Standardization (important before PCA)
- ☒ Centering data (standardized features)
- ☒ Computation of Covariance Matrix
- ☐ Choose component selection method (e.g., Scree, Cumulative Variance)
- ☐ Handle very large, sparse datasets ($O(n * d^2)$)

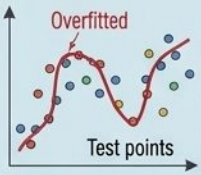
Dimensionality Reduction with PCA is powerful for compression and feature engineering, but interprstability is subjsctive and computationally intensive on largs d! :)

MODEL EVALUATION – OVERFITTING & UNDERFITTING



1. The Concept of Fit: Generalization Performance

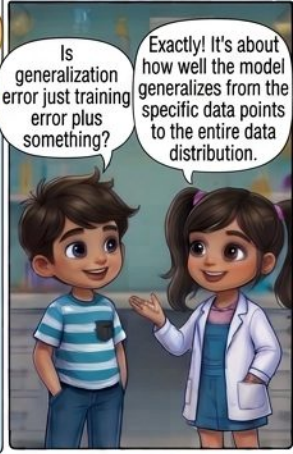


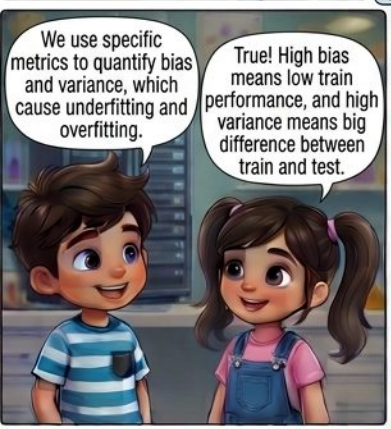


Goal: Find the sweet spot between training error and generalization error (test error).

High bias
High variance

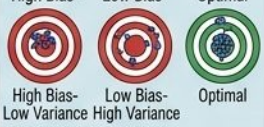
```python
import numpy as np
from sklearn.metrics import mean_squared_error
# Train/test split and performance check
mse_train = mean_squared_error(y_train, y_train_pred)
mse_test = mean_squared_error(y_test, y_test_pred)
mse_test = mean_squared_error(y_test, y_test_pred)
print(f'Train NSE: {mse_train}, Test MSE: {mse_test}')
```





2. The Mathematical Approach and Key Performance Indicators

Bias vs. Variance

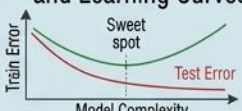


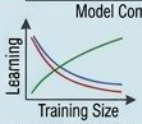
KPI: Mean Squared Error = $\sum x_6^2$

Bias and Variance Error = $1\mathcal{T}_0^2$

Bias and Variance decomposition

B KPI: Model Complexity and Learning Curves





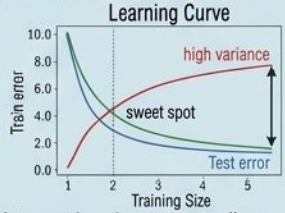
As complexity increases, train error drops, but test error might rise.

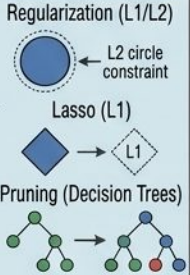
Performance is measured across model complexity to balance bias and. Monitor learning curves to find the optimal model complexity.





3. Mitigation Techniques and Interpreting Evaluation Results





Interpret learning curves to diagnose issues. A gap is high variance; poor convergence is high bias. Use techniques to correct.

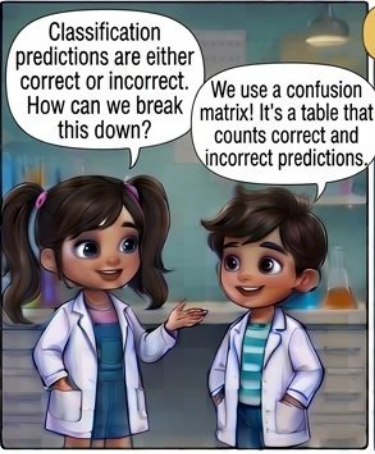
```python
from sklearn.linear_model import Ridge, Lasso

# Regularization techniques to mitigate overfitting
ridge = Ridge(alpha=1.0) # L2 penalty
lasso = Lasso(alpha=0.1) # L1 penalty
```

- [ ] Split data into Train and Test sets
- [x] Use Cross-Validation (e.g., K-fold)
- [x] Compute metrics for Bias and Variance
- [ ] Monitor learning curves
- [ ] Apply Regularization (L1/L2, pruning, early stopping)
- [ ] Collect more data

Model Evaluation is about identifying biss/variance trade-offs. Regularization and other metheds are powertui tor generalization, but tuning requires careful analysis! :)

MODEL EVALUATION – CONFUSION MATRIX



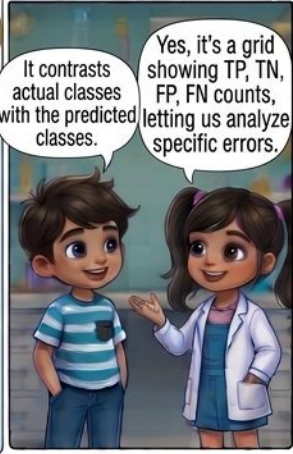
1. The Concept of Classification Results and the Matrix

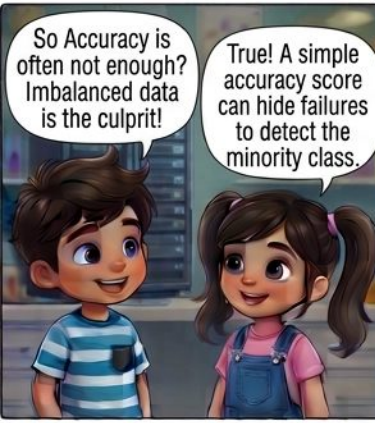
Actual \ Predicted	1	0
1	True Positive (TP) Example: 1	False Positive (FP, Type I error) Example: 1
0	False Negative (FN, Type II error) Example: 1	True Negative (TN) Example: 3

Goal: Analyze specific error types and calculate performance metrics like precision and recall.

High bias
High variance

```python
import numpy as np
from sklearn.metrics import confusion_matrix
# y_true and y_pred for a binary classification task
cm = confusion_matrix(y_true, y_pred)
print(f"Confusion Matrix:\n{cm}")
```





2. The Mathematical Approach and Key Performance Metrics

A Metric Formulas from the Matrix

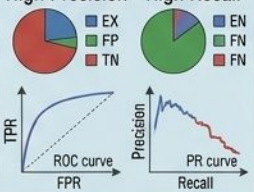
Accuracy: $\frac{TP + TN}{Total}$

Precision: $\frac{TP}{TP + FP}$

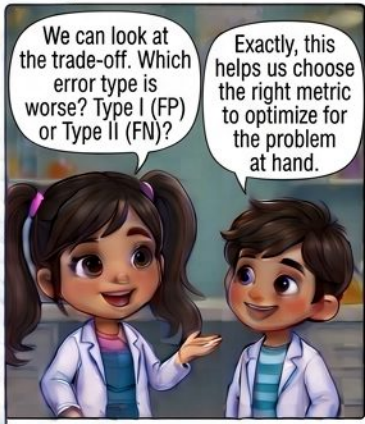
Recall (Sensitivity): $\frac{TP}{TP + FN}$

Specificity: $\frac{TN}{TN + FP}$

B KPI: Balancing Precision and Recall



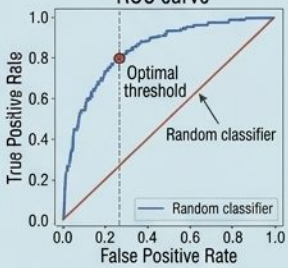
Precision and recall are trade-offs! Accuracy can be misleading with imbalanced data. F1-Score balances both.



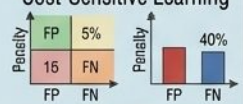


3. Mitigation Techniques and Interpreting Evaluation Results

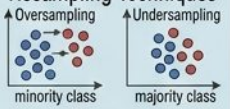
ROC curve



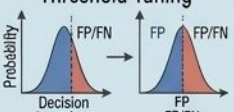
Cost-Sensitive Learning



Resampling Techniques



Threshold Tuning

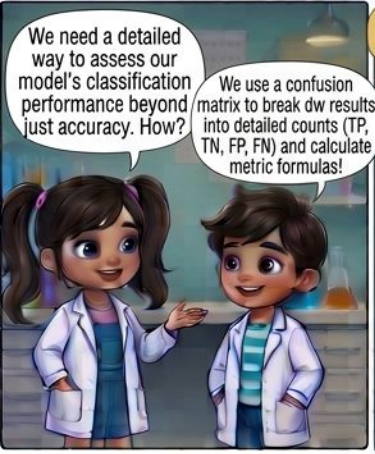


Interpret the ROC and PR curves. Adjust decision thresholds to optimize the metric you care about. Consider costs and balance.

- [ ] Create Confusion Matrix
- [x] Calculate basic counts (TP, TN, FP, FN)
- [x] Derive Metrics (Accuracy, Precision, Recall)
- [ ] Use Cross-Validation (e.g., K-fold for matrix metrics)
- [ ] Monitor ROC/PR curves
- [ ] Apply cost-sensitive learning or resampling
- [ ] Adjust prediction thresholds

Model Evalustion with Contusion Matrices reveals detailed error analysis. Optimizing with thresholds and cost-sensitive methods provides robust obust classification! :)

MODEL EVALUATION – KEY CLASSIFICATION METRICS

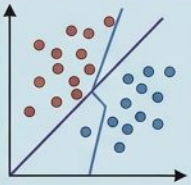


1. Basic Metrics from the Confusion Matrix: Accuracy and Precision

Accuracy:

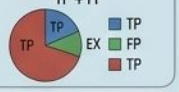
$$\text{Actor} = \frac{\text{Total correctness count}}{\text{Total predictions}} = \frac{TP + TN}{TP + TN + FP + FN}$$

Goal: Analyze general correctness, but can be misleading with imbalanced data.

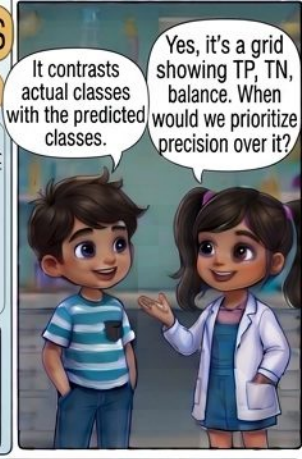


Precision:

$$\frac{\text{True Positive correctness count}}{\text{Positive predictions}} = \frac{TP}{TP + FP}$$




```python
import numpy as np
from sklearn.metrics import accuracy_score, precision_score,
            recall_score, f1_score, roc_auc_score, roc_curve
accuracy  = accuracy_score(y_true, y_pred)
precision = precision_score(y_true, y_pred)
print(f'Accuracy: {accuracy:.2f}, Precision: {precision:.2f}')
```





2. The Trade-Off Metrics and KPI: Recall, F1-Score, and Balancing

A Metric Formulas from the Matrix

$$\text{Accuracy: } \frac{TP}{TP + FN}$$



$$\text{Precision: } \frac{TP}{TP + FN}$$



Recall (Sensitivity):

$$\text{F1-Score: } 2\,\frac{(\text{Precision} \cdot \text{Recall})}{(\text{Precision} + \text{Recall})}$$

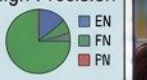
B KPI: Harmonic Mean

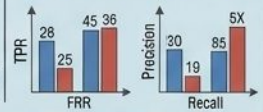
$$2 * (\text{Precision} \cdot \text{Recall})$$

High Recall



High Precision



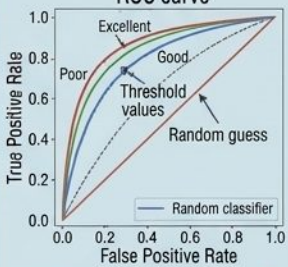


Recall captures all actual positives, but might have higher False Positives. Precision is correctness of positive predictions. F1-Score balances both.

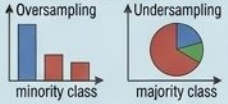




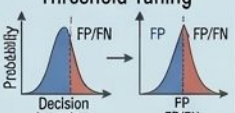
3. Advanced Evaluation and Choosing the Best Threshold: The ROC Curve



Goal: Visualize trade-offs between TPR and FPR across all decision thresholds. AUC score quantifies overall performance. Adjust threshold-ges to select the optimal point based on error costs.



Threshold Tuning



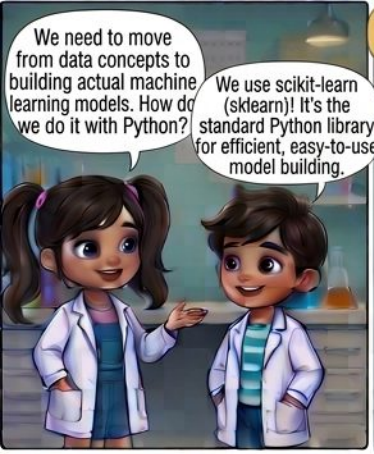
Interpret the ROC curve. Analyze AUC (Area Under the Curve). A perfect classifier has AUC=1.0. An AUC < 0.5 means the classifier is not useful. Adjust threshold selection and balance.

- [ ] Create Confusion Matrix
- [x] Calculate basic counts (TP, TN, FP, FN)
- [x] Derive Basic Metrics (Accuracy, Precision, Recall, F1-Score)
- [ ] Use Cross-Validation (e.g., K-fold for metric scores)
- [ ] Analyze performance across classes
- [ ] Monitor ROC Curve and AUC Score
- [ ] Select optimal prediction thresholds

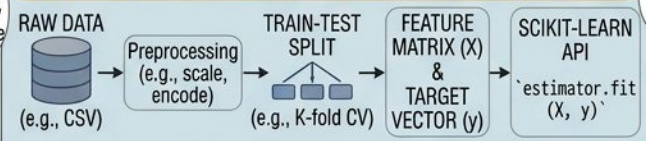


Comprehensive Model Evaluation with metrics, F1-Score, and ROC analysis reveals detailed trade-offs. Using AUC and choosing optimal thresholds provides reliable results! :)

INTRODUCTION TO SCIKIT-LEARN – BUILDING MODELS

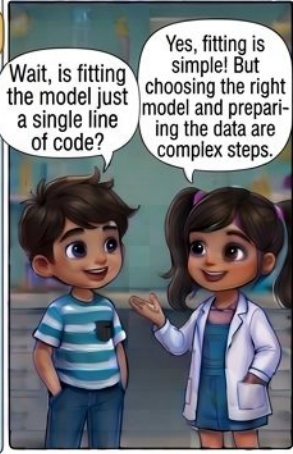


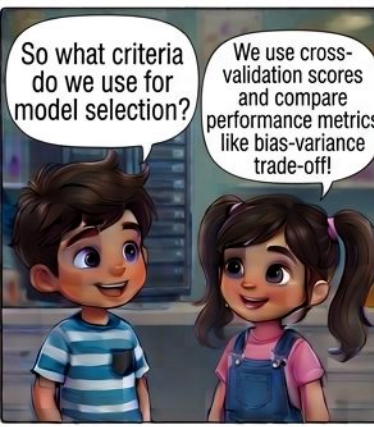
1. Building Models: The core workflow in scikit-learn




```python
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

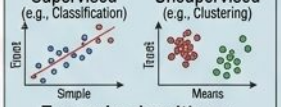
# Raw data processing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
# Instantiate estimator
clf = LogisticRegression()
# Fit model to data
clf.fit(X_train, y_train)
```





2. Key Concepts and Model Selection (KPI)

A) Model Types



Example algorithms:
Linear Regression, SVM, k-Means

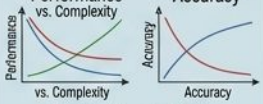
$$h(\mathbf{x}) = w^{T*}\mathbf{x} + b$$

$$h(\mathbf{x}) = w^{T*}\mathbf{x} + b$$



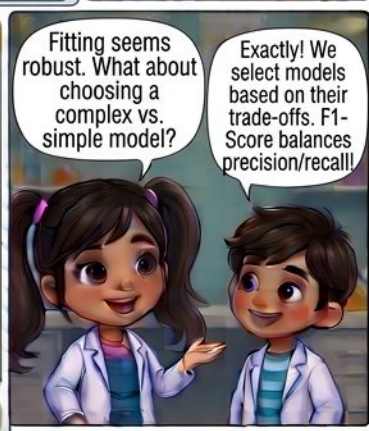
Scikit-learn has estimator classes for various tasks.

B) Model Evaluation & Selection



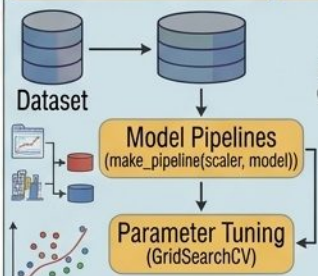


Goal: Balance model complexity with generalization error and select optimal parameters.

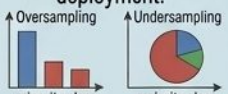


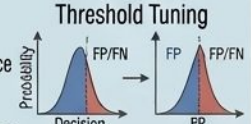
3. The Scikit-learn Ecosystem and Best Practices





Goal: Use Pipelines to streamline preprocessing and model training. Tuners optimize hyper-parameters. ROC analysis guides deployment.





Integrate tools. Understand performance trade-offs. Standardize workflow. Balance parameters and generalization. Choose threshold selection.

- [x] Create data for X, y matrix/vector
- [ ] Import estimator class (e.g., LinearRegression)
- [x] Split into Train and Test data
- [ ] Implement data preprocessing with Pipelines
- [ ] Select and instantiate model
- [ ] Perform Model Tuning and Cross-Validation
- [ ] Interpret evaluation and model results

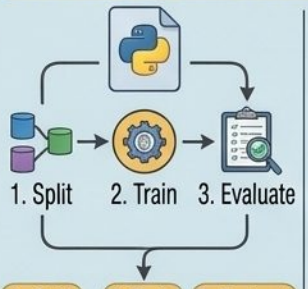
Introduction to Scikit-learn streamlines model building. Pipelinss and model tuning reveal trade-offs. Using Cross-Validation and choosing optimal estimators provides reliable results! :)

TRAINING AND EVALUATING ML MODELS IN PYTHON

We need to learn how to build and judge a good model!

Right! And for that, we need a complete Python workflow.

TRAINING & EVALUATION PIPELINE



Is there a difference in how we train and evaluate?

Yes! First, split data, then train, then test!

```python
# The standard scikit-learn approach
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

# 1. SPLIT: 80% train, 20% test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# 2. TRAIN: Fit to train data
model = LinearRegression()
model.fit(X_train, y_train)

# 3. EVALUATE: Predict on test data
y_test_pred = model.predict(X_test)
```

What metrics can we use?

It depends on the problem! For regression, MSE or R-Squared!

3. The Mathematical Approach and Key Performance Indicators

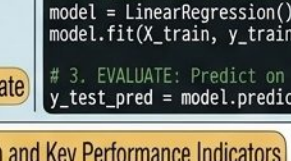
Regression Metrics:

MSE MAE R-Squared

For classification, use Accuracy, Precision, Recall!

Accuracy Precision Recall F1

k-Fold Cross-Validation (k=5)




```python
# K-Fold CV example
from sklearn.model_selection
import cross_val_score
```

```python
from sklearn.metrics import classification_report
# Compare predictions to ground truth
print(classification_report(y_test, y_test_pred))
```

And how do we get a more reliable estimate?

We use Cross-Validation to assess stability!

So we check learning curves?

Yes! Compare train vs. validation curves. The gap tells us about fit!

Learning Curve



Small gap = good generalization
Large gap = overfit

Okay, I've got the steps down!

```python
from sklearn.linear_model import Ridge, Lasso

# Regularization techniques to mitigate overfitting
ridge = Ridge(alpha=1.0) # L2 penalty
lasso = Lasso(alpha=0.1) # L1 penalty
```

- [ ] Split data (X_train, X_test, ...)
- [ ] Select and Instantiate a model
- [ ] Fit the model to X_train
- [ ] Make predictions on X_test
- [ ] Choose and Calculate relevant Metrics (e.g., MSE)
- [ ] Use Cross-Validation for robust scores
- [ ] Plot Learning Curves to check fit

Model evaluation is crucial! It ensures your model performs well on new data and you aren't just over-optimizing for the training set! :)